

AIM2- Preparations

DAY 2 — Python Collections

Today's topics:

1. Lists ★★★★★
2. Indexing
3. Slicing
4. Adding/removing items
5. Updating items
6. Tuples
7. Sets
8. Dictionaries ★★★★★
9. Looping through collections
10. Mini Project — **Student Gradebook**

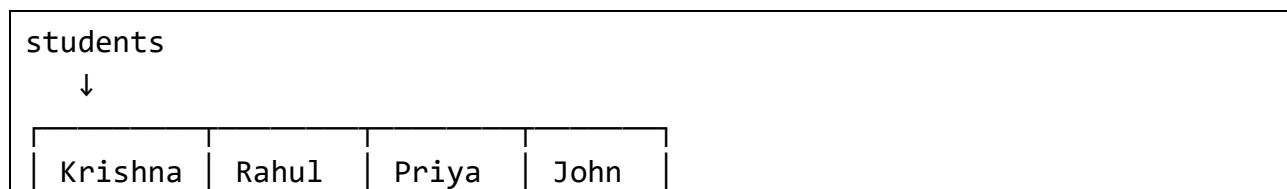
And I'm going to explain the **why**, not just the syntax.

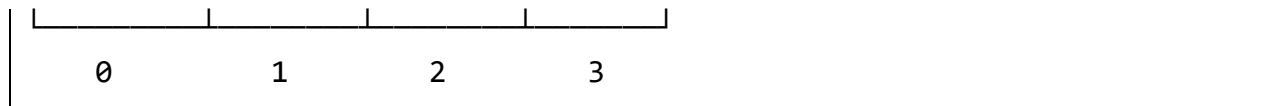
① Lists — Your most important collection today

A list stores multiple values in one variable.

```
students = ["Krishna", "Rahul", "Priya", "John"]
```

Think of it as:





The numbers underneath are called **indexes**.

2 Indexing

Python starts counting from **0**.

This is VERY important.

```
students = ["Krishna", "Rahul", "Priya", "John"]  
  
print(students[0])  
print(students[1])  
print(students[2])  
print(students[3])
```

Output:

Krishna
Rahul
Priya
John

So:

```
index 0 → first item  
index 1 → second item  
index 2 → third item  
index 3 → fourth item
```

Java connection

This is similar to a Java array:

```
String[] students = {"Krishna", "Rahul", "Priya", "John"};
```

```
System.out.println(students[0]);
```

Python:

```
students = ["Krishna", "Rahul", "Priya", "John"]
```

```
print(students[0])
```

Very similar.

3 Negative indexing 🧐

Python has a very convenient feature.

You can count from the end.

```
students = ["Krishna", "Rahul", "Priya", "John"]
```

```
print(students[-1])
```

Output:

John

Because:

```
-1 → last  
-2 → second last  
-3 → third last
```

Example:

```
print(students[-2])
```

Output:

Priya

This is something you'll use frequently in Python.

4 Lists can contain different types

Python lists don't have to contain only strings.

```
scores = [95, 88, 65, 92]
```

You can even have:

```
student = ["Krishna", 28, 89.67, True]
```

That's legal.

But in real applications, we usually structure our data more carefully.

Later, Pandas will let us work with columns such as:

Name	Age	Math	Python	AI
Krishna	28	95	88	65

So we're building toward that.

5 Changing an item

Lists are **mutable**.

That means we can change their contents.

```
students = ["Krishna", "Rahul", "Priya"]
```

```
students[1] = "Amit"
```

```
print(students)
```

Output:

```
['Krishna', 'Amit', 'Priya']
```

We changed:

Rahul → Amit

6 Adding items

append()

Adds an item to the end.

```
students = ["Krishna", "Rahul"]
```

```
students.append("Priya")
```

```
print(students)
```

Result:

```
['Krishna', 'Rahul', 'Priya']
```

Think:

```
append()
```

↓

ADD TO END

insert()

You can add something at a specific position.

```
students = ["Krishna", "Rahul"]
```

```
students.insert(1, "Priya")
```

```
print(students)
```

Result:

```
['Krishna', 'Priya', 'Rahul']
```

Because:

```
insert(index, value)
```

7 Removing items

remove()

```
students = ["Krishna", "Rahul", "Priya"]  
  
students.remove("Rahul")  
  
print(students)
```

Result:

```
['Krishna', 'Priya']
```

It removes the specified value.

pop()

```
students = ["Krishna", "Rahul", "Priya"]  
  
students.pop()  
  
print(students)
```

Result:

```
['Krishna', 'Rahul']
```

By default, `pop()` removes the last item.

You can also specify an index:

```
students.pop(1)
```

List length

Use:

`len()`

```
students = ["Krishna", "Rahul", "Priya", "John"]  
  
print(len(students))
```

Output:

4

This will become extremely important later.

Checking if something exists

Python gives us a very readable way:

```
students = ["Krishna", "Rahul", "Priya"]  
  
print("Krishna" in students)
```

Result:

True

And:

```
print("John" in students)
```

Result:

False

You can also use:

```
if "Krishna" in students:  
    print("Student found")
```

We'll use this kind of logic frequently.

10 List slicing

You learned string slicing yesterday.

The exact same concept works with lists.

```
students = ["Krishna", "Rahul", "Priya", "John", "Amit"]
```

```
print(students[0:3])
```

Result:

```
['Krishna', 'Rahul', 'Priya']
```

Remember:

start included

end excluded

So:


```
students[0:3]
```

means:

index 0 

index 1 

index 2 

index 3 

More examples

```
students[:3]
```

means:

Start from the beginning, take the first 3.

```
students[2:]
```

means:

Start at index 2 and go to the end.

```
students[:]
```

means:

Take the whole list.

Sorting

Suppose:

```
scores = [75, 95, 62, 88, 91]
```

You can sort:

```
scores.sort()
```

```
print(scores)
```

Result:

```
[62, 75, 88, 91, 95]
```

Descending:

```
scores.sort(reverse=True)
```

Result:

```
[95, 91, 88, 75, 62]
```



Let's practice before moving on

Create:

```
week-01-python/  
└─ exercises/  
    └─ day-01-fundamentals.py  
    └─ day-02-collections.py
```

Start day-02-collections.py.

Write your own examples for:

Exercise 1

Create a list containing **5 AIM2 subjects**.

Then:

- print the first subject
- print the last subject
- change the third subject
- add a new subject

- remove one subject
- print the total number of subjects

1 2 Tuples

Now we have another collection:

```
subjects = ("Python", "Machine Learning", "NLP", "Deep Learning")
```

It looks almost like a list.

But there is a major difference.

List:

```
subjects = ["Python", "Machine Learning", "NLP"]
```

Tuple:

```
subjects = ("Python", "Machine Learning", "NLP")
```

Lists use:

```
[ ]
```

Tuples use:

```
( )
```

Main difference

A list can be changed:

```
subjects = ["Python", "NLP"]
```

```
subjects[0] = "Machine Learning"
```

Works.

But:

```
subjects = ("Python", "NLP")
```

```
subjects[0] = "Machine Learning"
```

will produce an error.

A tuple is **immutable**.

Meaning:

Once created, its elements cannot be changed.

When do we use tuples?

When the data shouldn't change.

For example:

```
coordinates = (43.65, -79.38)
```

or:

```
rgb = (255, 255, 255)
```

or:

```
student_id = ("AIM2", 2026)
```

For now, remember:

List → can change

Tuple → cannot change

Don't overthink tuples yet.

1 3 Sets

Now let's meet:

```
skills = {"Python", "Java", "Python", "Docker"}
```

What do you think happens?

```
print(skills)
```

You'll get something like:

```
{'Python', 'Java', 'Docker'}
```

Notice:

Python appeared only once.

That's because a **set does not store duplicate values.**

Example

```
numbers = {1, 2, 2, 3, 3, 3, 4}
```

```
print(numbers)
```

Result:

```
{1, 2, 3, 4}
```

This makes sets extremely useful when you need **unique values.**

List vs Tuple vs Set

Here's your first important comparison:

Collection	Syntax	Ordered	Duplicates	Changeable
List	[]	Yes	Yes	✓
Tuple	()	Yes	Yes	✗
Set	{ }	No guaranteed indexing/order	✗	✓

Don't worry about memorizing every detail yet.

The big idea:

LIST

"I have a collection that I may modify."

TUPLE

"I have a fixed collection."

SET

"I care about unique values."

★ 1 4 Dictionaries — VERY IMPORTANT

This is the collection I especially want you to understand today.

A dictionary stores:

KEY → VALUE

For example:

```
student = {  
    "name": "Krishna",  
    "age": 28,  
    "program": "AIM2",  
    "math": 95,
```

```
"python": 88,  
"ai": 65  
}
```

Think of it like:

```
student  
├── name    → Krishna  
├── age     → 28  
├── program → AIM2  
├── math    → 95  
├── python  → 88  
└── ai      → 65
```

This is much more meaningful than:

```
student = ["Krishna", 28, "AIM2", 95, 88, 65]
```

Because with the list, you have to remember:

```
index 0 = name  
index 1 = age  
index 2 = program  
...
```

With a dictionary:

```
student["name"]
```

is obvious.

1 **5** Access dictionary values

```
student = {  
    "name": "Krishna",  
    "age": 28,
```

```
        "program": "AIM2"
    }

    print(student["name"])
    print(student["age"])
    print(student["program"])
```

Output:

Krishna
28
AIM2

1 6 Add a new key

```
student["city"] = "London"
```

Now:

```
print(student)
```

contains:

city → London

1 7 Update a value

```
student["age"] = 29
```

The old value:

28

becomes:

29

1 8 Remove a dictionary item

```
student.pop("city")
```

Now city is removed.

1 9 Check if a key exists

```
if "name" in student:  
    print("Name exists")
```

This is useful when processing real-world data.

★ 2 0 Dictionary + List

Now we're getting closer to **real data**.

You can have a list of dictionaries:

```
students = [  
    {  
        "name": "Krishna",  
        "math": 95,  
        "python": 88,  
        "ai": 65  
    },  
    {  
        "name": "Rahul",  
        "math": 82,  
        "python": 91,  
    }  
]
```

```
        "ai": 78
    },
    {
        "name": "Priya",
        "math": 90,
        "python": 94,
        "ai": 89
    }
]
```

Now we have:

students



Student 1 dictionary
Student 2 dictionary
Student 3 dictionary

We can access:

```
print(students[0]["name"])
```

Result:

Krishna

And:

```
print(students[1]["python"])
```

Result:

91

This structure is extremely important.

Because later you'll encounter data conceptually like:

CSV
↓
Rows
↓
Records
↓
Columns

And then Pandas will make working with this much easier.

Loops — First introduction

Now suppose we want to print every student.

We don't want:

```
print(students[0])  
print(students[1])  
print(students[2])
```

Instead:

```
for student in students:  
    print(student)
```

This means:

For every item inside `students`, temporarily call it `student` and execute the indented code.

You might get:

```
{'name': 'Krishna', 'math': 95, 'python': 88, 'ai': 65}  
{'name': 'Rahul', 'math': 82, 'python': 91, 'ai': 78}  
{'name': 'Priya', 'math': 90, 'python': 94, 'ai': 89}
```

We will go **much deeper into loops**, so don't worry if this feels new.

2 2 Loop through dictionary values

You can do:

```
for student in students:  
    print(student["name"])
```

Output:

Krishna

Rahul

Priya

And:

```
for student in students:  
    print(f"{student['name']}: {student['python']}")
```

Output:

Krishna: 88

Rahul: 91

Priya: 94

🔥 This is the kind of Python thinking we want to develop.

DAY 2 MINI PROJECT

Now we're going to make your Day 1 project more realistic.

Yesterday you had:

ONE student

Today we're going to create:

MULTIPLE students

Student Gradebook

Create:

```
week-01-python/  
└─ exercises/  
    └─ day-02-collections.py
```

Start with:

```
students = [  
    {  
        "name": "Krishna",  
        "math": 95,  
        "python": 88,  
        "ai": 65  
    },  
    {  
        "name": "Rahul",  
        "math": 82,  
        "python": 91,  
        "ai": 78  
    },  
    {  
        "name": "Priya",  
        "math": 90,  
        "python": 94,  
        "ai": 89  
    }  
]
```

Your task is to write code that prints:

```
=====
      STUDENT GRADEBOOK
=====
```

Name: Krishna
Math: 95
Python: 88
AI: 65

Name: Rahul
Math: 82
Python: 91
AI: 78

Name: Priya
Math: 90
Python: 94
AI: 89

Then calculate for EACH student:

Total
Average
Result

For example:

Name: Krishna
Math: 95
Python: 88
AI: 65
Total: 248
Average: 82.67
Result: PASSED